



HEI-Vs Engineering School - Industrial Automation Base

Cours AutB

Author: [Cédric Lenoir](#)

Version 2026, V1.0

LAB 01 Introduction à la programmation

Annexe:

- [QuickStart_ctrlX_PLC](#) to load and upload your first project from archive.

Keywords: IDE HMI NODE-RED FUNCTION BLOCK CYCLIC TASK R_TRIG TON

Objectifs

Acquérir des connaissances des différents systèmes de développement utilisés pendant le semestre et se familiariser avec les bases de la programmation PLC et de l'interface Node-RED.

Afin d'atteindre ces objectifs, les exercices suivants seront effectués :

- Programmation d'un détecteur de front
- Programmation d'une temporisation
- Programmation d'un signal sinusoïdal

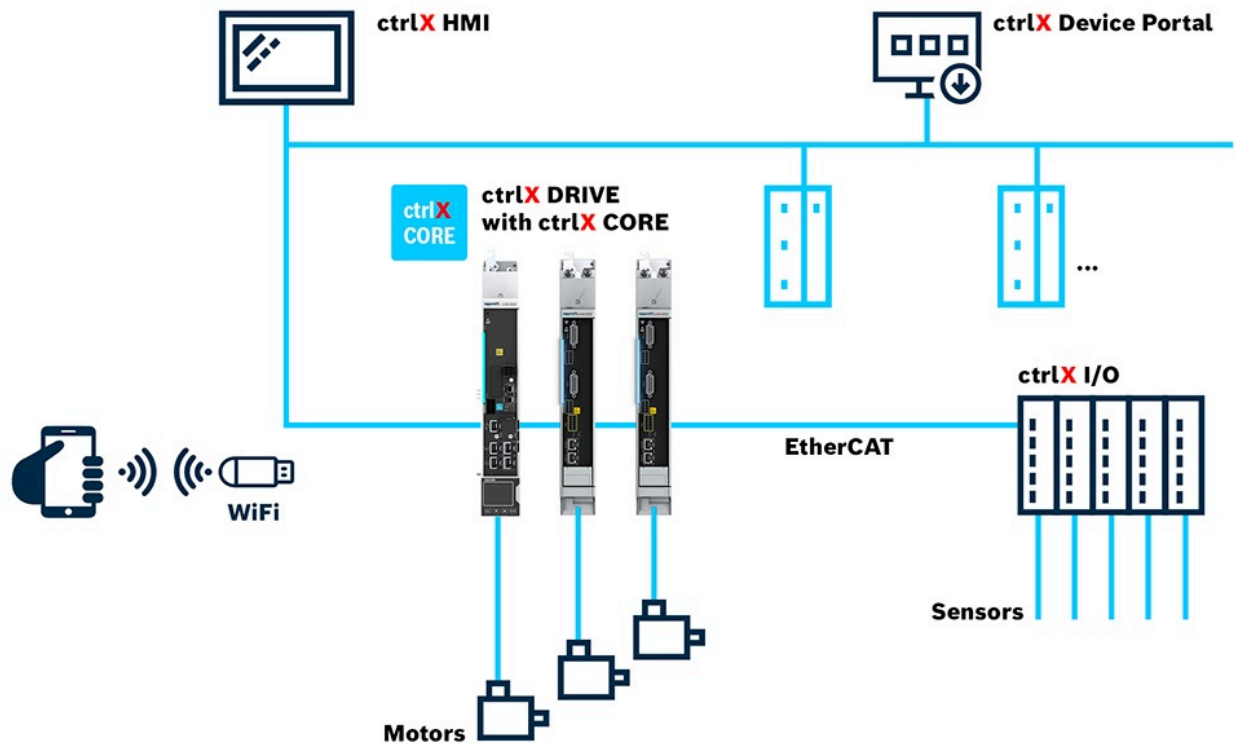
- Programmation d'une interface sur Node-RED pour lire et écrire des variables depuis/vers le PLC

Présentation de l'environnement

CtrlX Core

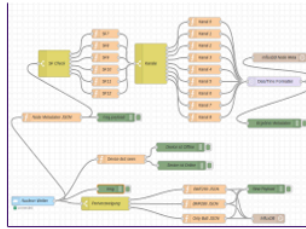
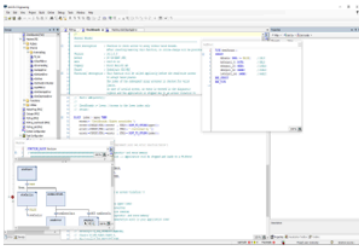
Le programme PLC sera exécuté dans le **ctrlX Core** à disposition sur chaque unité du laboratoire d'automatisation.

Le **ctrlX Core** fonctionne sur la base d'un système d'exploitation Linux "temps réel" embarqué dans une commande électrique d'axe (Processeur 64 Bit Quad Core ARM).



CtrlX CORE Architecture drive based

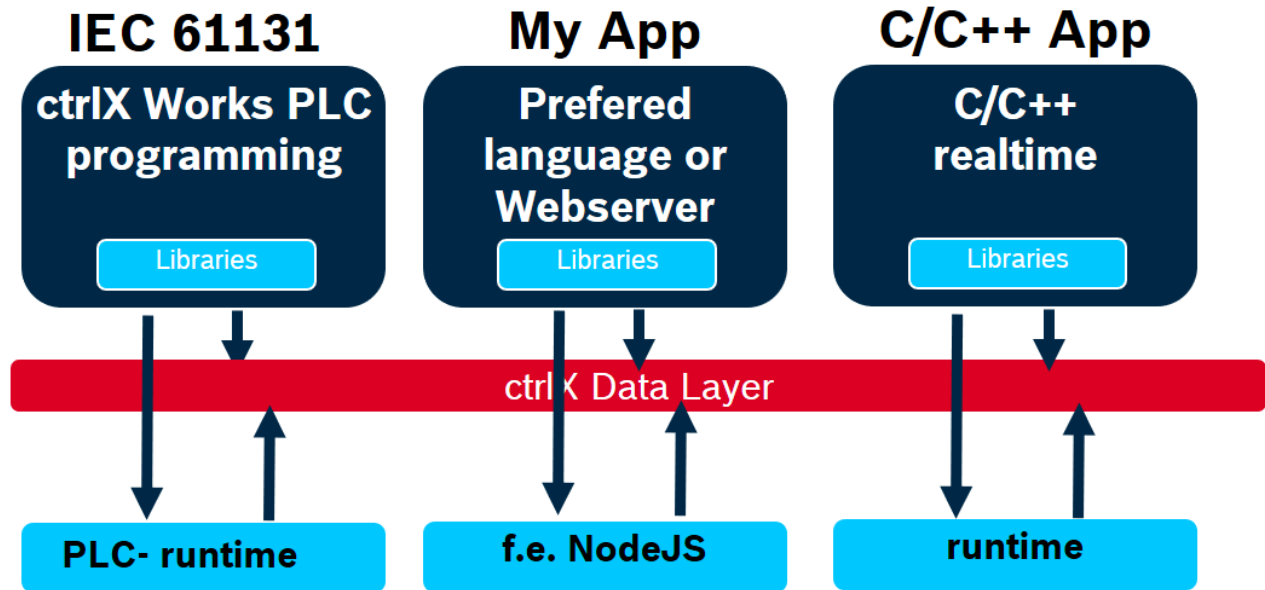
Le "ctrlX Core" est une architecture ouverte qui permet l'intégration de diverses applications et services.



```

let [tracks : getTracks (java.sound.midi.MidiSystem.getMessageSequence (new java.io.File filename))]
track (from tracks)
track (to tracks tracknum)
len (size track)
tempo (reduce +
  loop (for [1
    let [message : getMessage (.get track i)]
    data (getdata message)
    if (== (gettype message) 32)
      (* (byteint (data i)) 128000)
      (* (byteint (data i)) 128000) (byteint (data i))
    )
  ])
  (range (size track)))
]
loop [1 8
  tune [1]
  opens [1]
  let [evt (.get track i)
    msg (.getMessage evt)
    and (if (contains java.sound.midi.ShortMessage msg) (.getCommand msg))
    pitch (if end (.getdata msg))
  ]

```



ctrlX CORE PLC Runtime Overview

Logiciels

ctrlX WORKS

CtrlX WORKS est une suite logicielle développée par Bosch Rexroth pour programmer et gérer les systèmes **ctrlX Core**.

Elle permet la gestion des appareils, qu'ils soient réels ou virtuels, connectés au PC de développement.

ctrlX WORKS inclut un environnement de développement intégré (IDE) pour la programmation selon les langages décrits dans la norme IEC 61131-3 qui se nomme **ctrlX PLC Engineering**.

Version de CtrlX WORKS utilisée lors de l'écriture de ce document : 3.6.3

Lien pour télécharger l'environnement de développement :

[Full development environment for ctrlX OS](#)



ctrlX WORKS - Device Management



ctrlX PLC Engineering

HMI :

Les HMI (**Human Machine Interface**) actuels sont souvent développées avec des technologies WEB, HTML, Java Script qui ne sont pas des technologies qui font partie du programme SYND, Systèmes Industriels.

Actuellement, de plus en plus de solutions de HMI sont basées sur des notions de **Low Code** ou **No Code**. La solution proposée dans le cadre des travaux pratiques utilisera l'environnement **Node-Red** qui est une solution **Low Code**. Elle offre la possibilité d'accéder aux données du PLC avec un minimum de code.

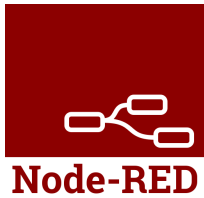


Exemple avec un Dashboard de Node-RED

Node-Red

Node-RED est un outil de développement basé sur le flux pour la programmation visuelle, principalement utilisé pour connecter des dispositifs matériels, des API et des services.

Il permet de créer des applications en reliant des blocs de construction appelés "nœuds" dans une interface utilisateur graphique.



Quelques caractéristiques clés de Node-RED :

- Programmation visuelle : Utilise une interface glisser-déposer pour créer des flux de données.
- Basé sur Node.js : Ce qui permet d'utiliser une grande variété de modules Node.js.
- Extensible : Bibliothèque de nœuds régulièrement actualisée avec de nouvelles fonctionnalités.
- Open source : Gratuit et maintenu par une communauté active.

Installation de Node-Red :

Prérequis :

Afin que Node-RED puisse fonctionner, il est nécessaire d'avoir installé préalablement l'environnement d'exécution JavaScript "Node.js".

--> [Download Node.js](#)

Remarque : L'installation de cet environnement a déjà été effectuée sur les PC du laboratoire d'automatisation.

Par contre, **vous devez installer "Node-RED" sur le PC du laboratoire d'automatisation** en suivant la procédure ci-après.

A la suite de cette installation, votre environnement Node-RED sera installé dans votre propre profil utilisateur sous C:\Users\Your_Username\.node-red.

Procédure d'installation :

1) Installer Node-RED

- Ouvrir "l'invite de commande" (cmd.exe)
- Entrer la commande : `npm install -g --unsafe-perm node-red`

2) Démarrage de Node-RED

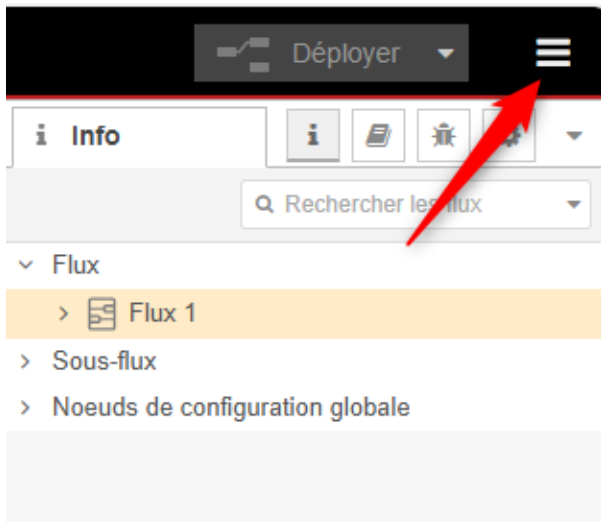
- Ouvrir l'invite de commande (cmd.exe)
- Entrer la commande : `cd C:\Users\Your_Username\AppData\Roaming\npm`
- Entrer la commande : `node-red`

3) Accéder à l'interface utilisateur de Node-RED

- Ouvrir le navigateur
- Entrer l'URL : <http://localhost:1880>

4) Installer les modules nécessaires à notre application

- Cliquer sur les 3 traits horizontaux en haut à droite



--> Gérer la palette

--> Sélectionner l'onglet "Installer"

--> Installer les modules suivants :

- `@flowfuse/node-red-dashboard`
- `node-red-contrib-ctrlx-automation`

5) Arrêter Node-RED

Dans la fenêtre de l'invite de commande, appuyer sur `Ctrl + C` ou fermer la fenêtre de l'invite de commande

N.B. : Après avoir démarré Node-RED pour la première fois, les fichiers de configuration et les flux seront stockés sous : C:\Users\Your_Username\.node-red.

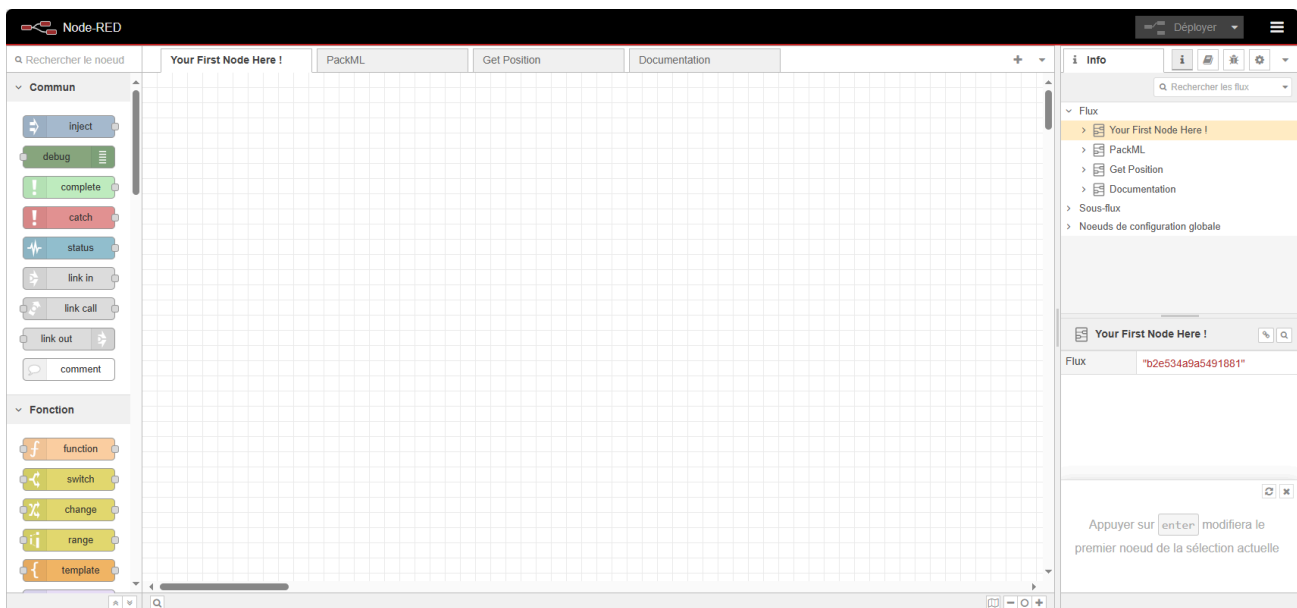
6) Copier le fichier de base "flows.json" mis à disposition

Remplacer le fichier "flows.json" qui se trouve dans le répertoire C:\Users\Your_Username\.node-red par le fichier "flows.json" mise à disposition dans Github.

7) Redémarrer Node-Red --> voir point 2)

8) Accéder à l'interface utilisateur de Node-RED --> voir point 3)

Vous devriez obtenir l'interface utilisateur suivante :



Votre premier programme

Objectif :

Implémenter un compteur dans une tâche et vérifier que cette dernière est exécutée cycliquement.

Prérequis :

Ouvrir ctrlX PLC Engineering et effectuer la procédure suivante :

--> [Procédure pour charger l'archive du projet ici.](#)

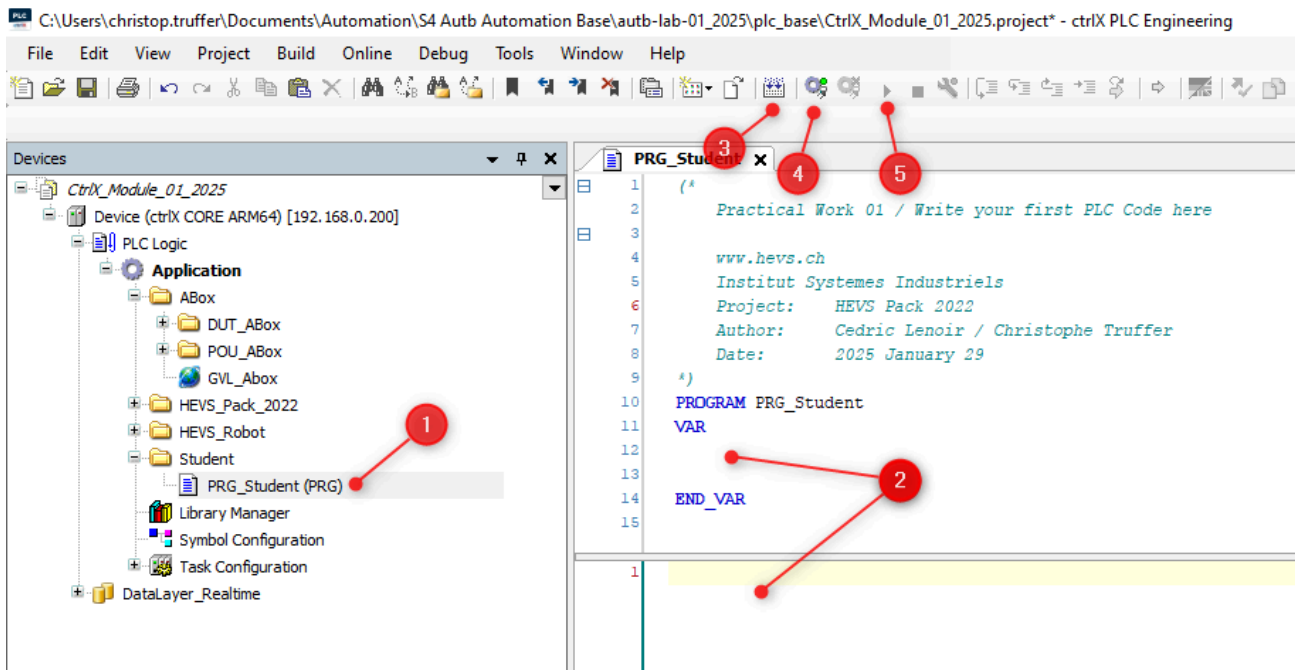
Implémenter le programme suivant dans le PLC

```
PROGRAM PLC_PRG
VAR
    iCount    : UINT := 3;
END_VAR

iCount := iCount + 1;
```

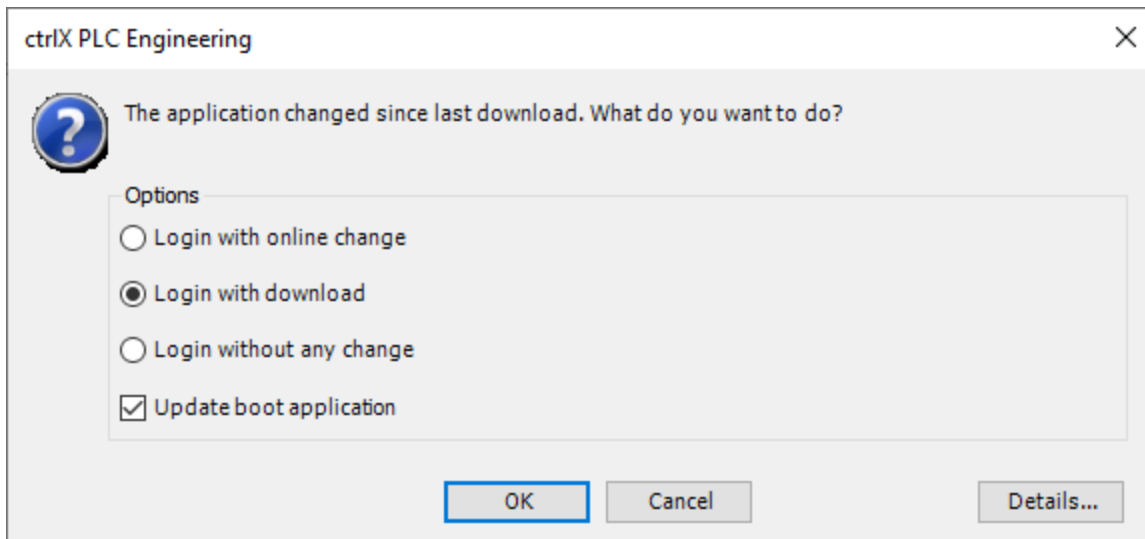
Marche à suivre :

1. Sélectionner PRG_Student (PRG).
2. Ecrire le programme.
3. Compiler.
4. Charger le programme dans le PLC.
5. Démarrer le programme.



Mon premier programme

La partie supérieure de la fenêtre permet de définir les variables et la partie inférieure à écrire le programme.

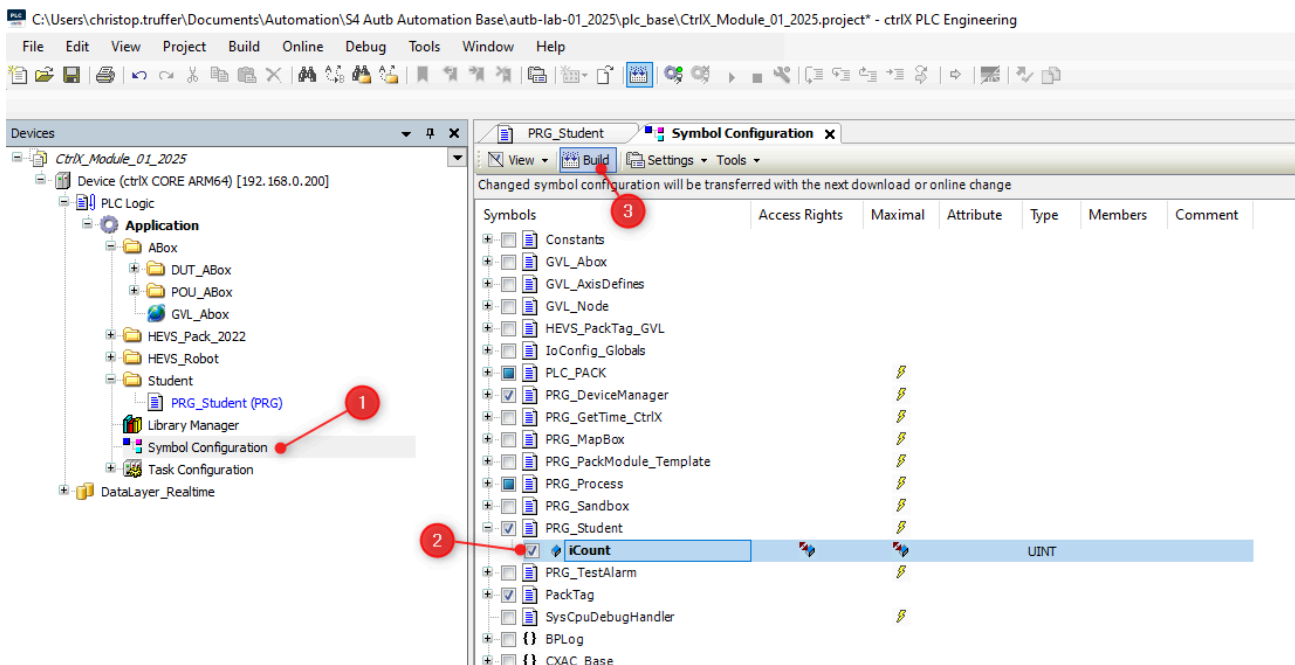


Sélectionner "Login with download"

Pour qu'une variable soit accessible depuis Node-RED, il faut en informer le compilateur.

Marche à suivre :

1. Sélectionner **Symbol Configuration**
2. Sélectionner la variable "iCount"
3. Cliquer sur "Build".



Configure Symbol Configuration

Si l'icône **Symbol Configuration** n'est pas présente, il faut l'ajouter en sélectionnant dans le menu Project --> Add Object --> Symbol Configuration...

Lors du prochain téléchargement du programme, la variable "iCount" sera accessible dans le **Data Layer** (*) du CtrlX-Core depuis Node-RED.

(*) Le Data Layer agit comme une colonne vertébrale dans l'architecture ctrlX CORE assurant l'échange de données "temps réel" et "non temps réel" entre les différentes applications.

Le Data Layer ne stocke pas directement les données, mais agit comme un intermédiaire connaissant leur emplacement et leur structure.

Implémenter le programme suivant dans Node-RED

1. Démarrer Node-RED

- Ouvrir l'invite de commande (cmd.exe)
- Entrer la commande : `cd C:\Users\Your_Username\AppData\Roaming\npm`
- Entrer la commande : `node-red`

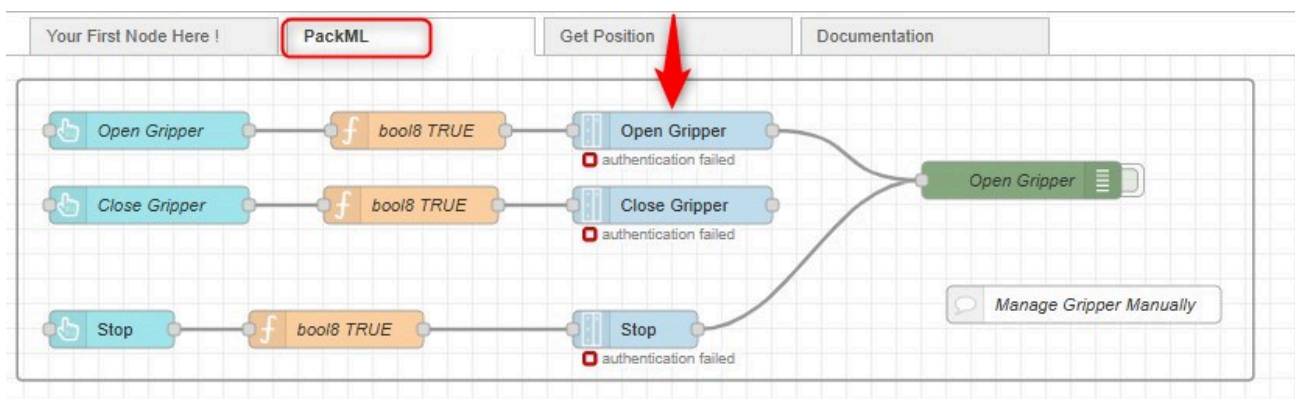
2. Accéder à l'interface utilisateur de Node-RED

- Ouvrir le navigateur
- Entrer l'URL : <http://localhost:1880>

3. Connecter Node-RED avec le "ctrlX Core"

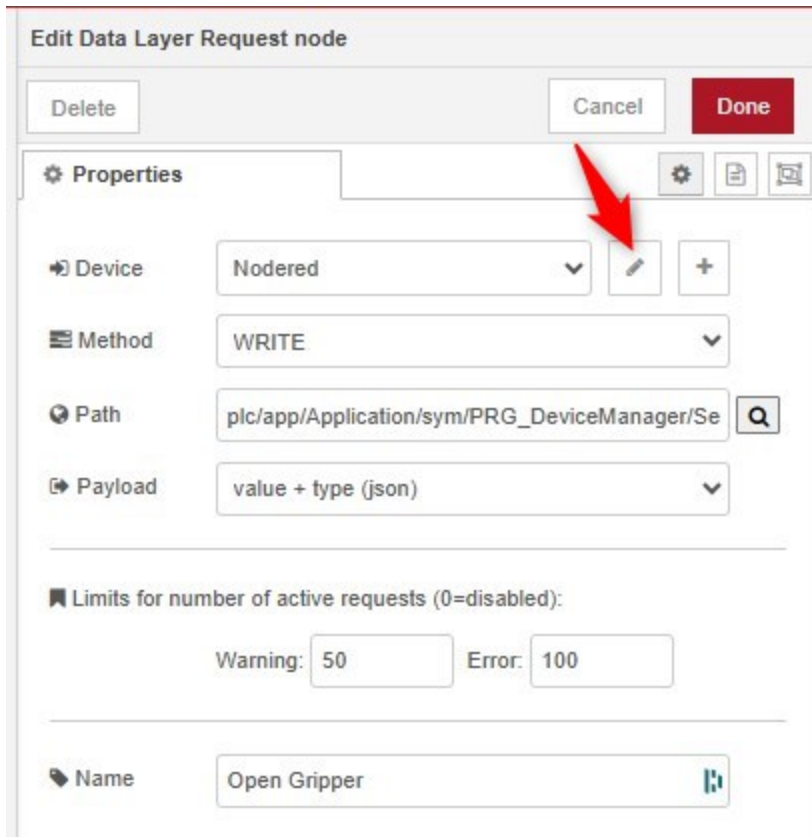
Procédure :

- Sélectionner l'onglet "PackML"
- Effectuer un double-clic sur le noeud "Open Gripper"



Noeud Open Gripper

- Cliquer sur le crayon à droite du champ "NodeRed"



The screenshot shows the 'Edit Data Layer Request node' dialog box. At the top, there are buttons for 'Delete', 'Cancel', and 'Done'. Below this is a 'Properties' section with several fields: 'Device' (set to 'Nodered'), 'Method' (set to 'WRITE'), 'Path' (set to 'plc/app/Application/sym/PRG_DeviceManager/Se'), and 'Payload' (set to 'value + type (json)'). A red arrow points to the edit icon (pencil) next to the 'Device' field. Below the 'Properties' section, there is a section for 'Limits for number of active requests (0=disabled):' with input fields for 'Warning: 50' and 'Error: 100'. At the bottom, there is a 'Name' field set to 'Open Gripper'.

Propriétés noeuds Open Gripper

- Compléter les différents champs comme suit :

Edit Data Layer Request node > Edit ctrlx-config node

Delete Cancel Update

Properties

Address 192.168.0.200

Username boschrexroth

Password

Name Nodered

☐ Debug mode

You need to configure a user and password for your ctrlX CORE connection.

Hint: You can configure a user and password after logging into your ctrlX CORE. It is recommended to configure a dedicated user for your Node-RED connection. Assign only the permissions, that you really need.

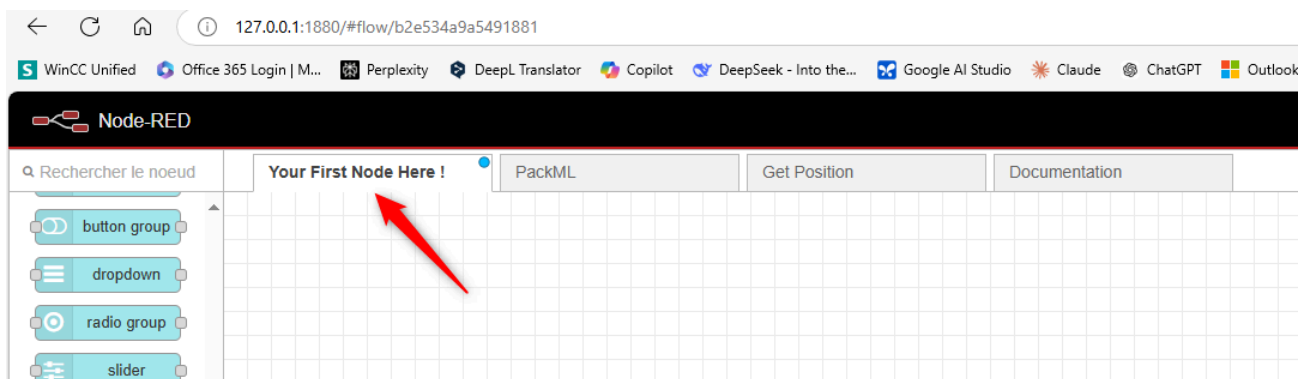
Champs noeud Open Gripper

Address : 192.168.0.200

Username : boschrexroth

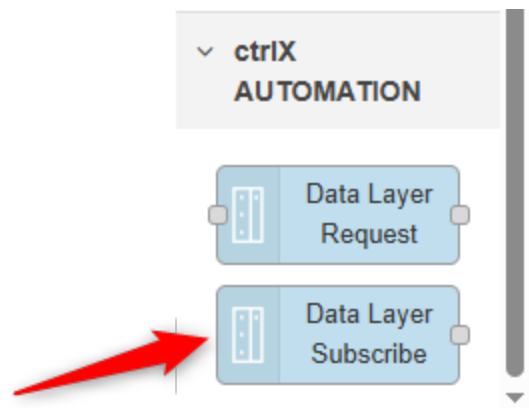
Password : Industrie_21

4. Accéder à l'onglet "Your First Node Here !"



Your first node

Ajouter le noeud **Data Layer Subscribe** et le configurer comme suit :



Sélection noeud DataLayer Subscribe

- Cliquer sur le bouton "+" à droite de "Subscription"
- Dans "Device", sélectionner "Nodered"
- Dans "Name", ajouter un nom (par exemple : iCount)

Propriétés

Device: Nodered

Publish Interval: use default Seconds

Sampling Interval: use default Seconds

Error Interval: use default Seconds

Keep-alive Interval: use default Minutes

Queue Size: use default Discard Oldest

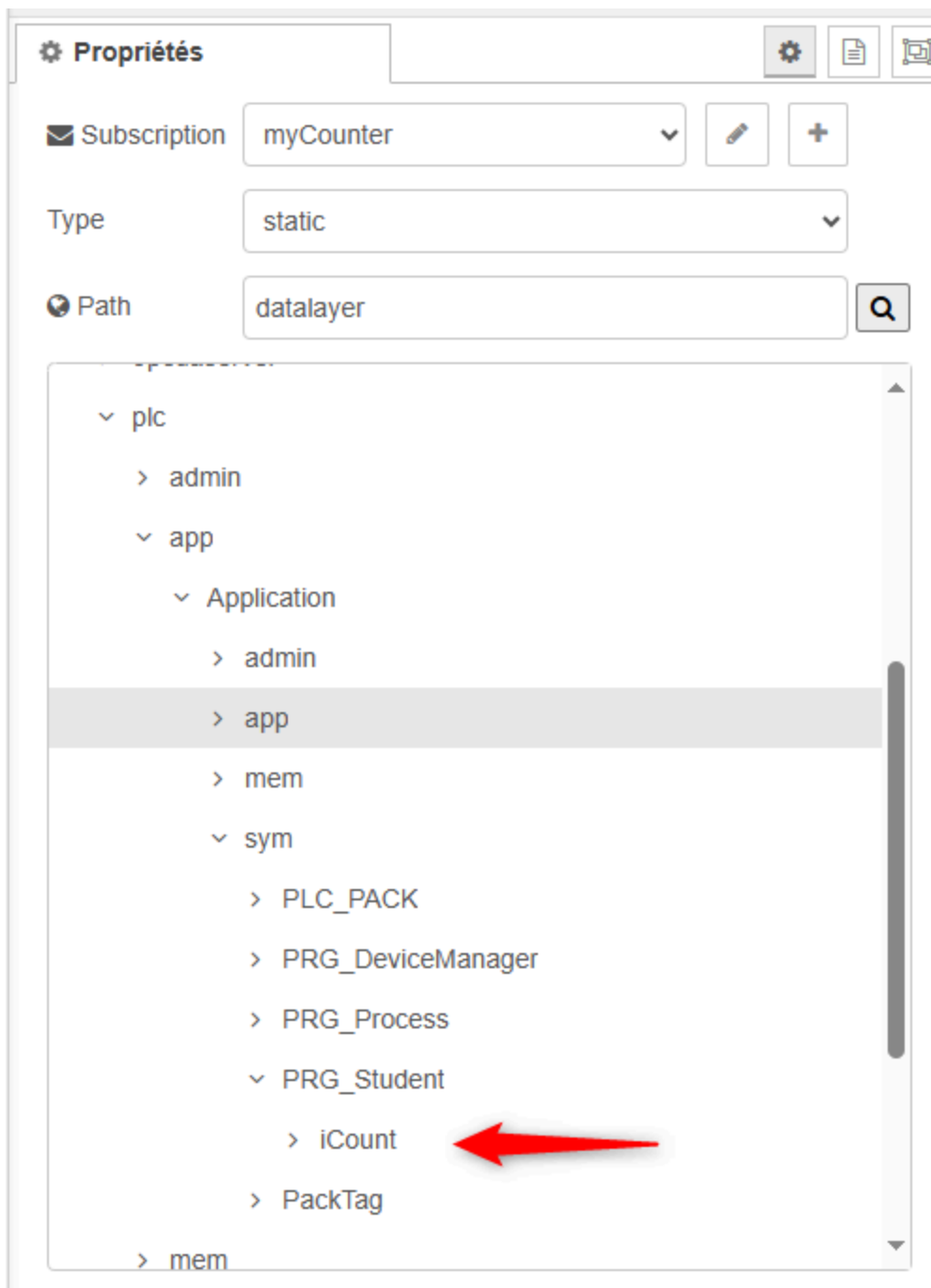
Deadband Value: 0.0

Name: myCounter

Hint: If no value is set, then the server side default is used.

Propriétés node "DataLayer Subscribe"

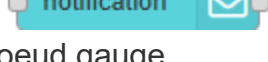
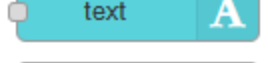
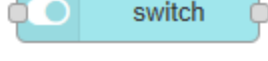
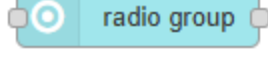
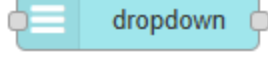
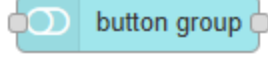
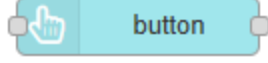
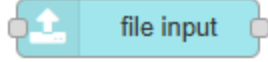
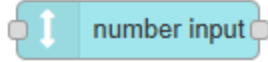
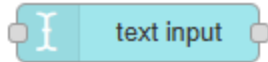
Dans "Path", sélectionner "iCount".



Propriétés "node DataLayer Subscribe"

Ajouter le noeud **gauge** et le configurer comme suit :

▼ dashboard 2



Sélection noeud gauge

Propriétés

Name

Group [Your First UI Page] Your First

Size 3 x 3

Type Half Gauge

Style Needle

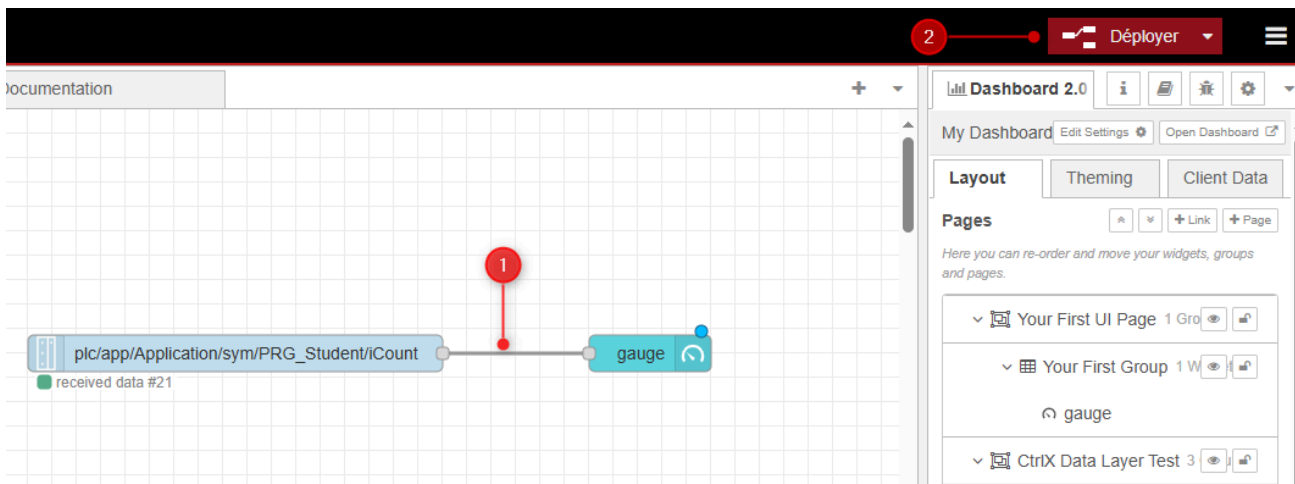
Limits

Range min. max.

Segments

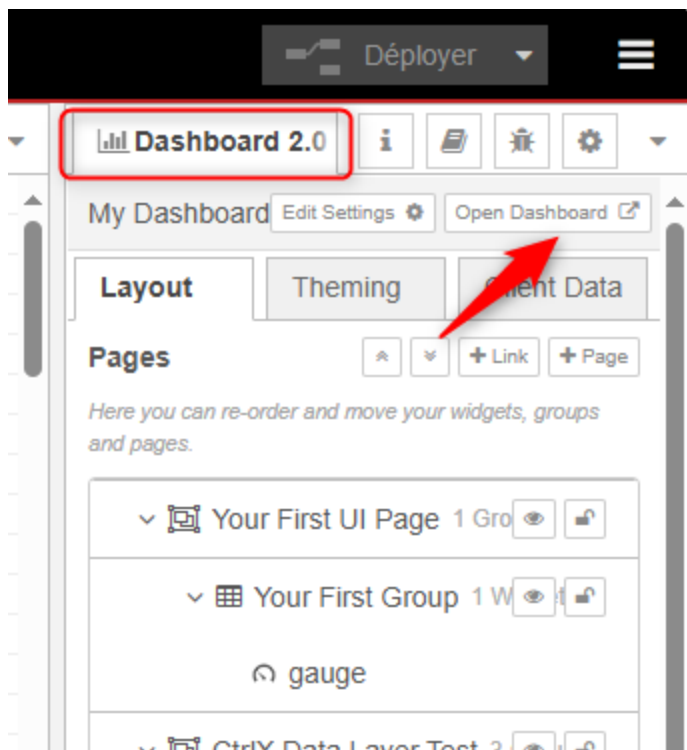
Propriétés noeud gauge

Relier ensuite ces 2 noeuds (1) et cliquer sur le bouton "Déployer" (2).



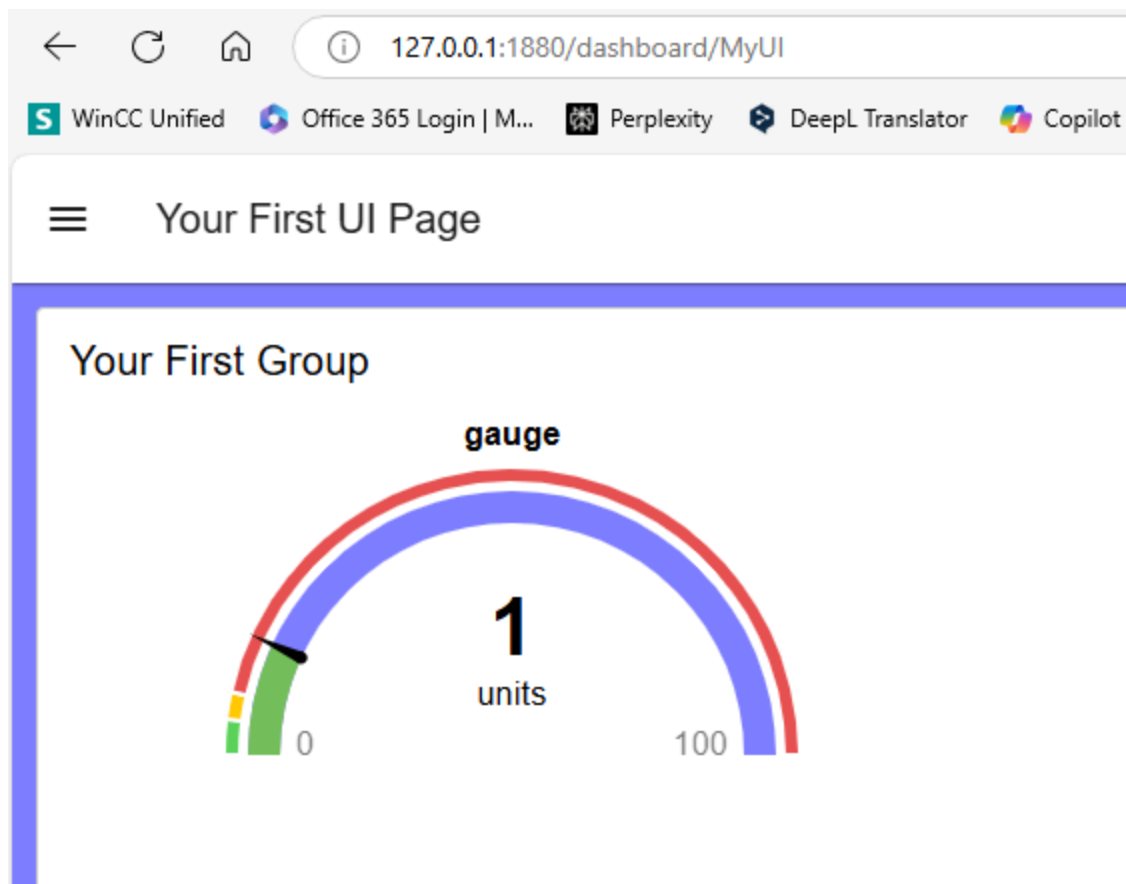
Noeud reliés ensemble

Démarrer le Dashboard



Démarrage du Dashboard

Résultat :



Affichage du compteur sur le Dashboard

Well done, your first program is ready !

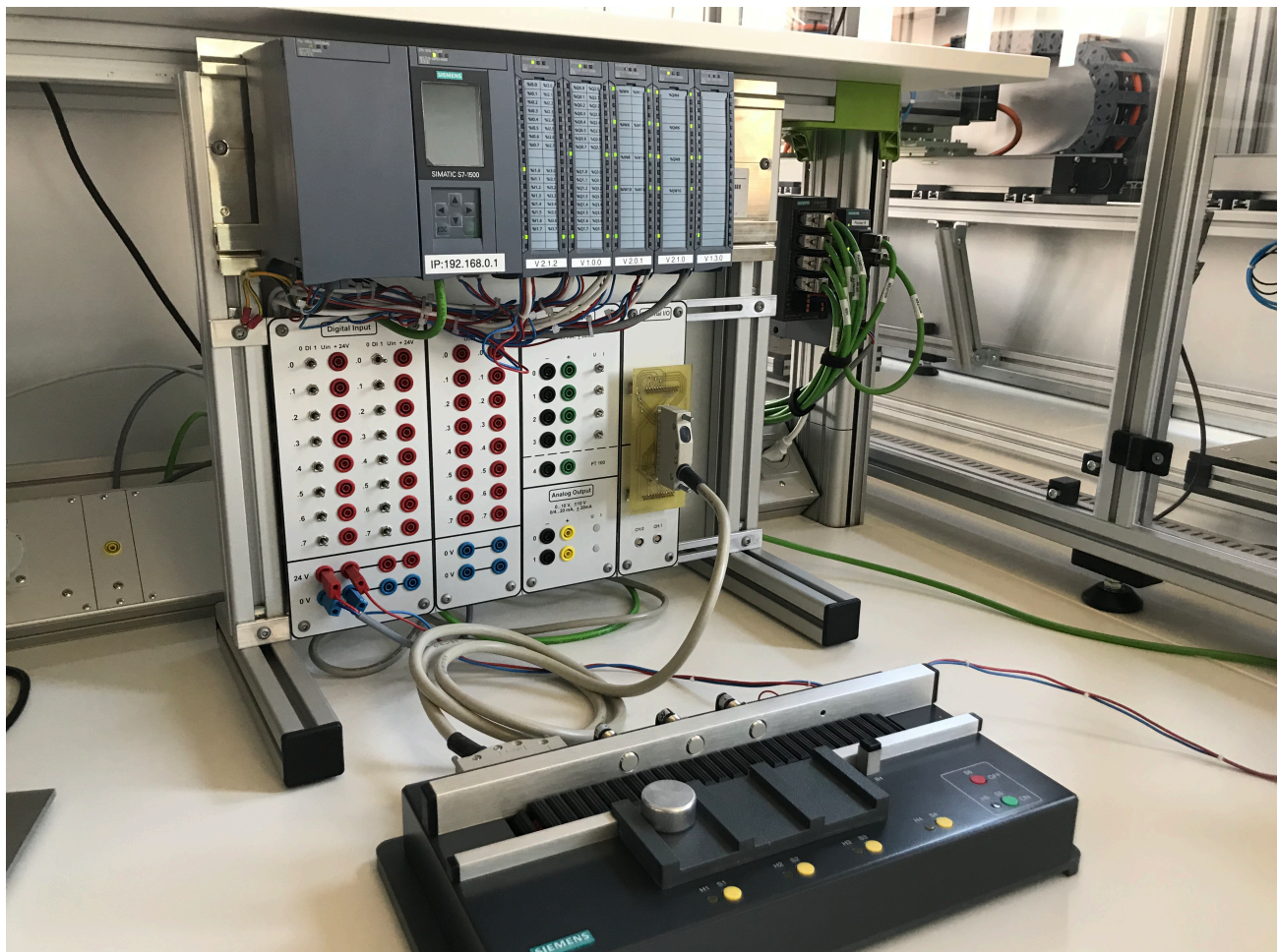
Programmer un détecteur de front

Objectif :

Démarrer le convoyeur dès qu'un front montant est détecté sur un bouton **start** et de l'arrêter dès qu'un front montant est détecté sur un bouton **stop**. Les deux boutons seront implémentés sur Node-RED.

Prérequis :

- Brancher le convoyeur au PLC "Siemens"
- Enclencher la maquette avec le bouton poussoir "S5"



Le convoyeur

Préambule :

La détection des fronts montants ou descendants peuvent être programmés en utilisant le bloc fonctionnel **R_TRIG** (Rising Trigger), respectivement le bloc fonctionnel **F_TRIG** (Falling Trigger).

Principe de fonctionnement de R_TRIG :

Ce bloc fonctionnel est constitué de :

- 1 entrée : CLK (BOOL)
- 1 sortie : Q (BOOL)

Lorsqu'un front montant est détecté sur l'entrée CLK, la sortie Q passe à TRUE pendant un seul cycle d'exécution du programme.

Compléter le programme PLC avec le code ci-dessous

```
PROGRAM PRG_Student
VAR
    iCount : UINT := 3;
    startFromNodeRed : BOOL;
    stopFromNodeRed : BOOL;
    rTrigStart      : R_TRIG;
    rTrigStop      : R_TRIG;

END_VAR
```

```

// Start conveyor
IF rTrigStart.Q THEN
    GVL_Abox.uaAboxInterface.uaDigitalOut.Output_0_4 := TRUE;
END_IF

// Stop conveyor
IF rTrigStop.Q THEN
    GVL_Abox.uaAboxInterface.uaDigitalOut.Output_0_4 := FALSE;
END_IF

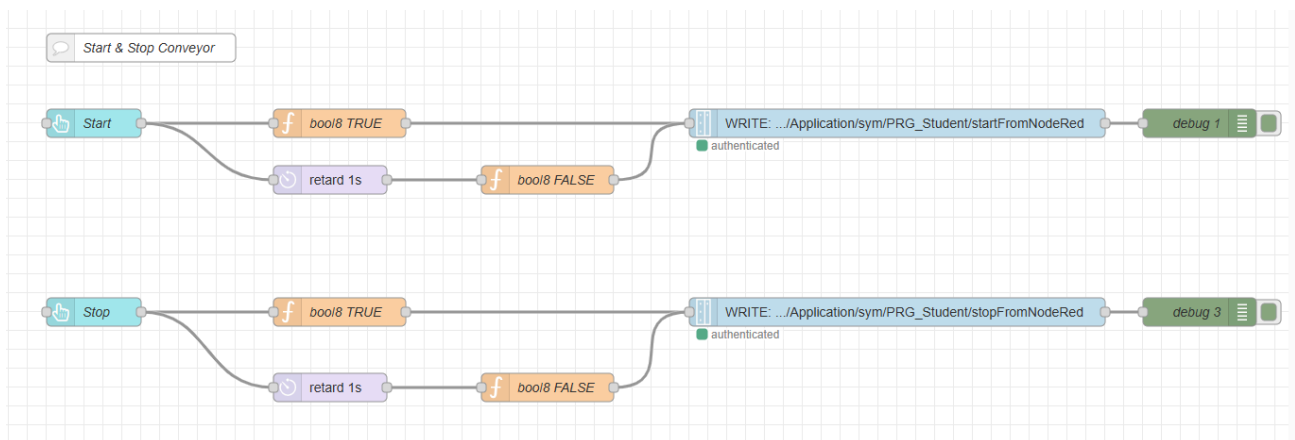
// Call function block
rTrigStart(CLK:=startFromNodeRed);
rTrigStop(CLK:=stopFromNodeRed);

```

N.B. : La variable structurée "GVL_Abox.uaAboxInterface.uaDigitalOut.Output_0_4" est affectée à la commande du moteur "direction droite" de la bande de transport du convoyeur.

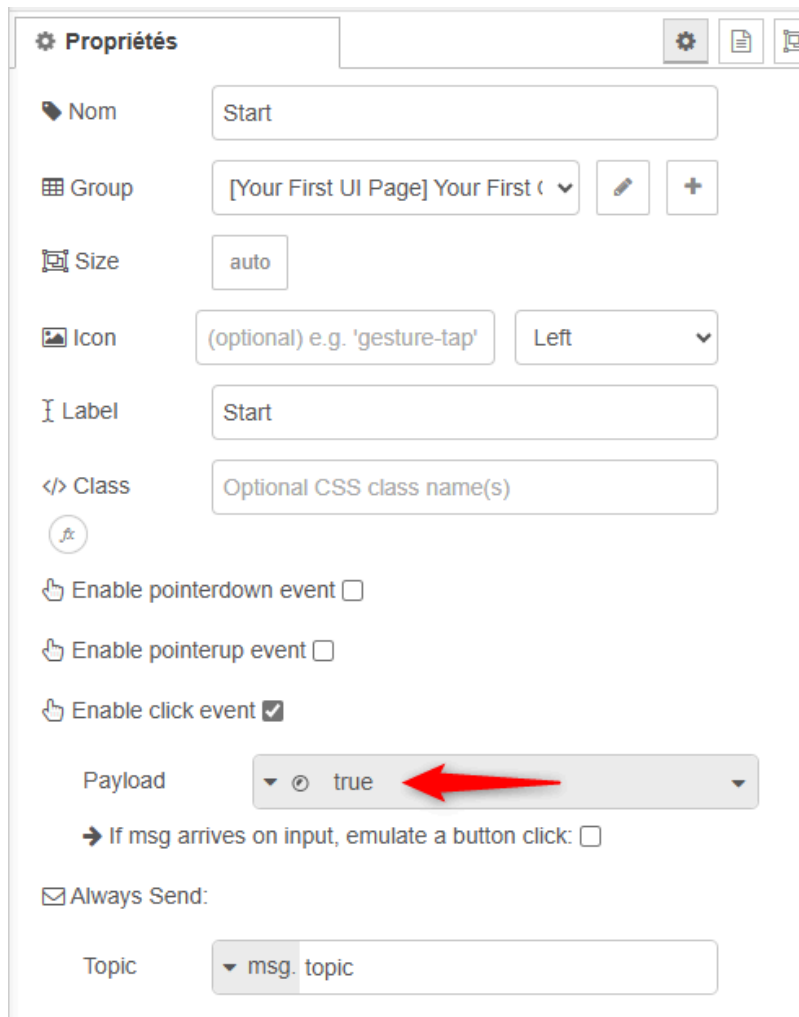
Il faut ensuite donner l'accès aux variables "startFromNodeRed" et "stopFromNodeRed" dans "Symbol Configuration" afin que Node-RED puisse les utiliser (Marche à suivre : voir exercice précédent).

Compléter le programme Node-RED avec le code ci-dessous



Start & Stop convoyeur

Propriété des boutons.



Propriétés

Nom: Start

Group: [Your First UI Page] Your First (v)

Size: auto

Icon: (optional) e.g. 'gesture-tap' Left (v)

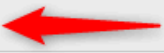
Label: Start

Class: Optional CSS class name(s)

Enable pointerdown event ☐

Enable pointerup event ☐

Enable click event ☒

Payload: true (v) 

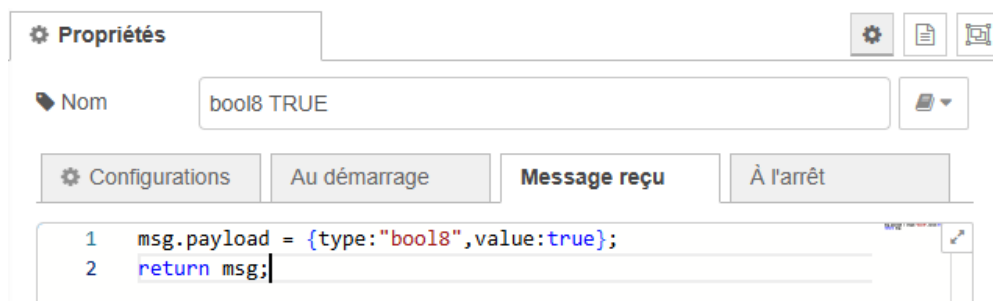
→ If msg arrives on input, emulate a button click: ☐

Always Send: ☒

Topic: msg.topic

Propriétés des noeuds Button

Propriété de la fonction "bool8 TRUE".



Propriétés

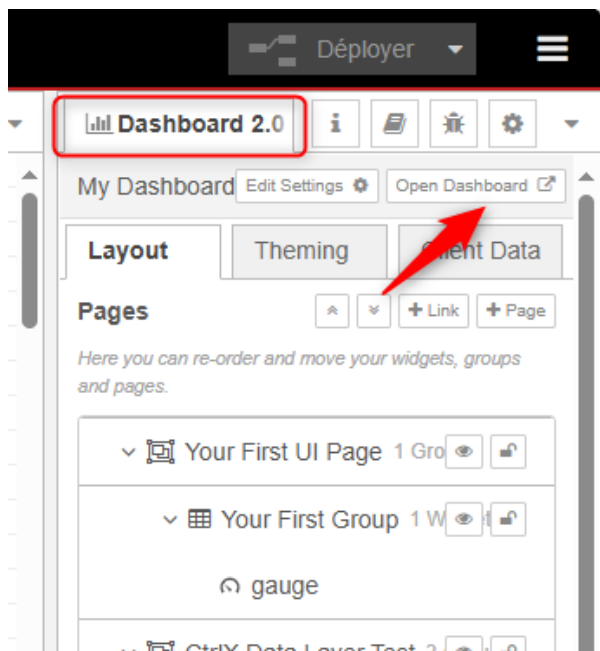
Nom: bool8 TRUE

Configurations | Au démarrage | **Message reçu** | À l'arrêt

```
1 msg.payload = {type:"bool8",value:true};
2 return msg;
```

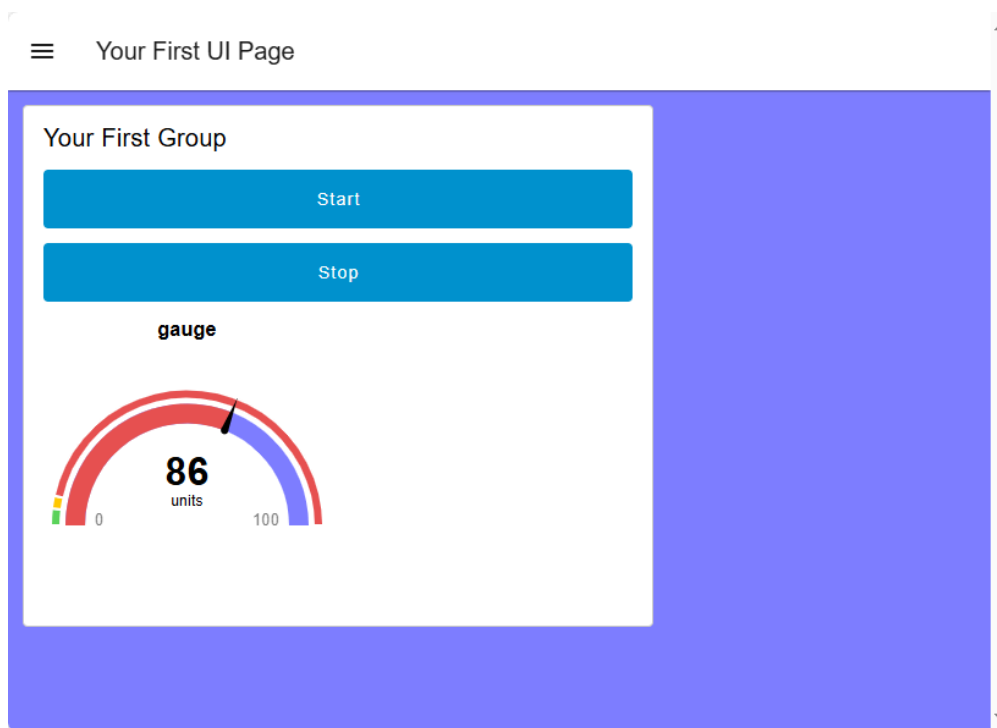
Propriétés de la fonction "bool8 TRUE"

Démarrer le Dashboard



Démarrage du Dashboard

Résultat :



Dashboard avec boutons "Start" & "Stop" convoyeur

Programmer une temporisation

Objectif :

Compléter le programme afin que le convoyeur s'arrête si le bouton **stop** a été appuyé ou si un **certain laps de temps s'est écoulé**.

Préambule :

Les temporisations sont des blocs fonctionnels très couramment utilisés dans la programmation **PLC**. On distingue les 3 types suivants :

1. **TON** (Timer On-Delay): C'est une temporisation à l'enclenchement. Elle active sa sortie après un délai programmé lorsque son entrée est activée.
2. **TOF** (Timer Off-Delay): Il s'agit d'une temporisation au déclenchement. Elle maintient sa sortie active pendant un temps défini après que son entrée soit désactivée.
3. **TP** (Timer Pulse): Cette temporisation génère une impulsion de durée fixe. Une fois déclenchée par un front montant sur son entrée, elle active sa sortie pour une durée déterminée, indépendamment des changements ultérieurs de l'entrée.

Chaque type de temporisation possède les paramètres suivants:

- IN: Entrée booléenne pour déclencher la temporisation
- PT: Durée programmée (Preset Time)
- Q: Sortie booléenne
- ET: Temps écoulé (Elapsed Time)

Les paramètres "PT" et "ET" ont un type de donnée **TIME**.

Exemple du format : T#5d4h3m2s1ms

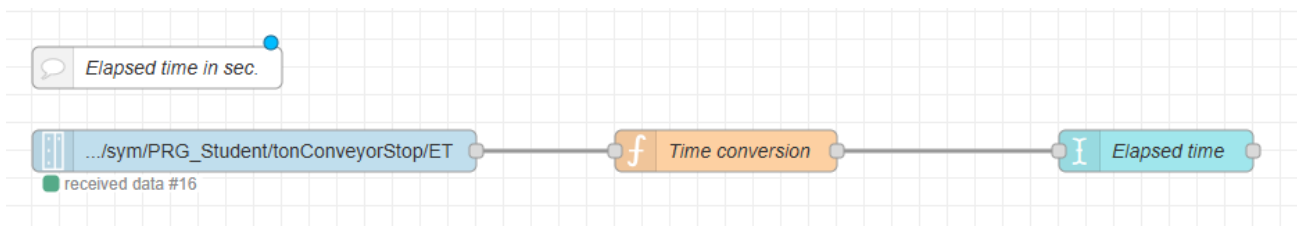
Compléter le programme PLC en ajoutant la déclaration de la variable ci-dessous

```
PROGRAM PRG_Student
VAR
    ...

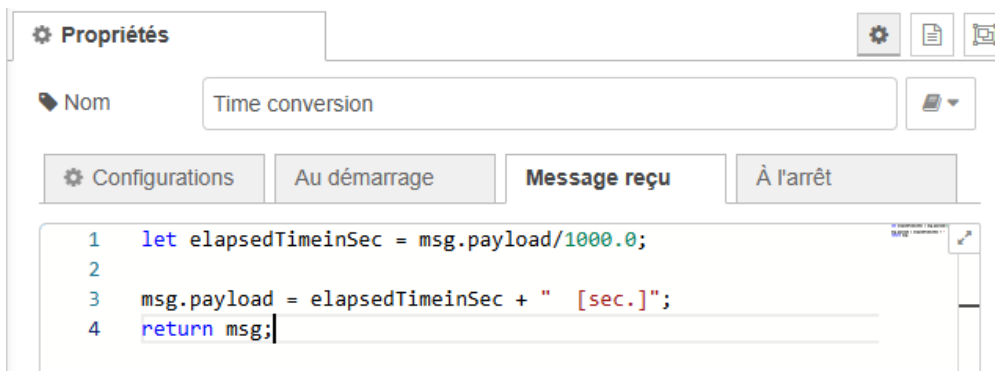
    tonConveyorStop : TON;
END_VAR
```

Compléter le programme Node-RED avec le code ci-dessous

L'objectif est d'afficher le temps écoulé de la temporisation "TON" sur Node-RED.

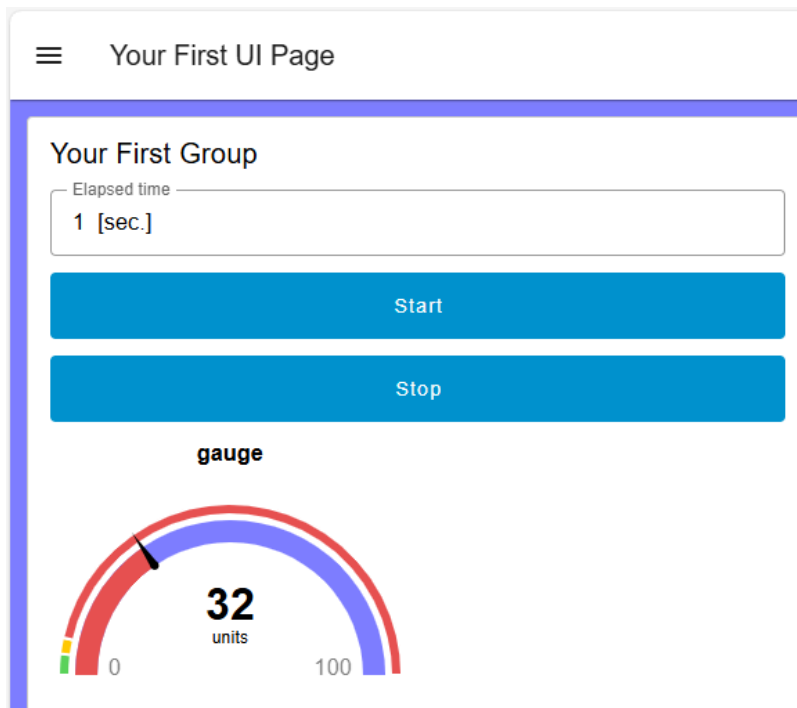


Temps écoulé en secondes



Propriétés du noeud "Time conversion"

Résultat :



Durée écoulée de la temporisation TON"

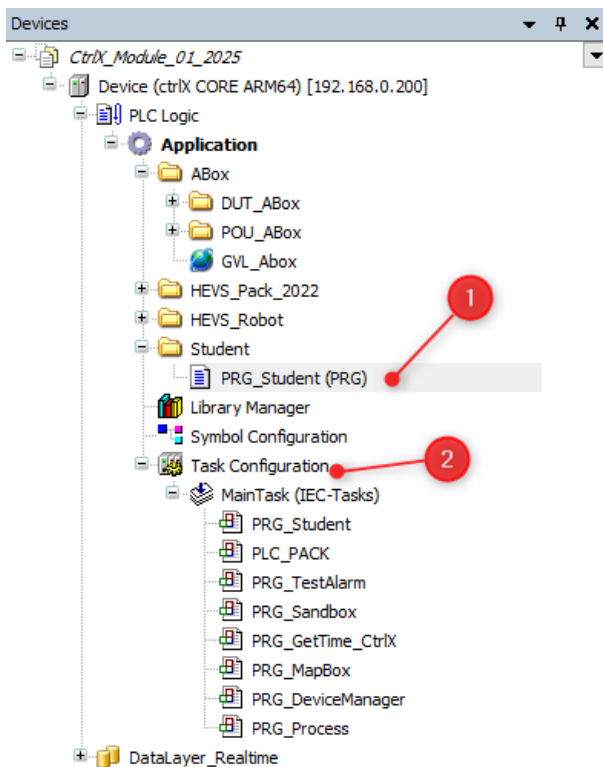
Programmer un générateur de signal sinusoïdal.

Objectif :

Compléter le programme en ajoutant la génération d'un signal sinusoïdal et son affichage sur un graphique.

Préambule :

Le programme "PRG" (1) est appelé cycliquement par la tâche "MainTask" (2).



Appel du programme "PRG" par la tâche "MainTask"

Vérifier le temps de cycle de la tâche principale "MainTask" qui devrait être de 40 [ms] .

POU	Comment
PRG_Student	
PLC_PACK	
PRG_TestAlarm	
PRG_Sandbox	
PRG_GetTime_CtrlX	
PRG_MapBox	
PRG_DeviceManager	
PRG_Process	

Temps de cycle de la tâche principale

Compléter le programme PLC en ajoutant la déclaration des variables ci-dessous

Données de base :

- Amplitude du signal : 10 [-] .
- Fréquence du signal : 0,1 [Hz] .
- Pi est défini comme constante.

```
PROGRAM PRG_Student
```

```
VAR
```

```
...
```

```
amplitude : REAL := 10.0;  
frequency_Hz : REAL := 0.1;  
sampleTime_s : REAL := 0.04;  
t_s : REAL := 0.0;  
sinusSignal : REAL;
```

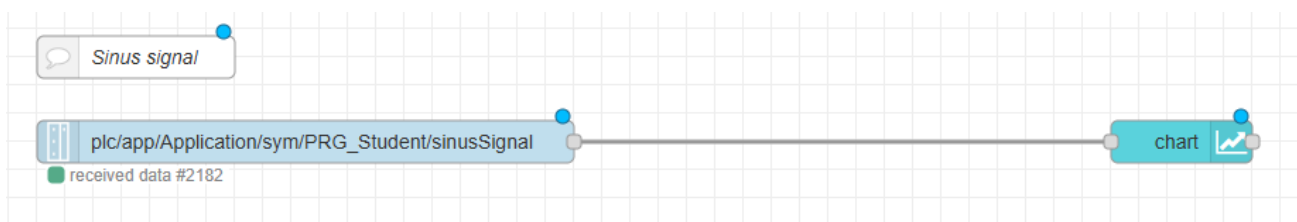
```
END_VAR
```

```
VAR CONSTANT
```

```
PI : REAL := 3.14159;
```

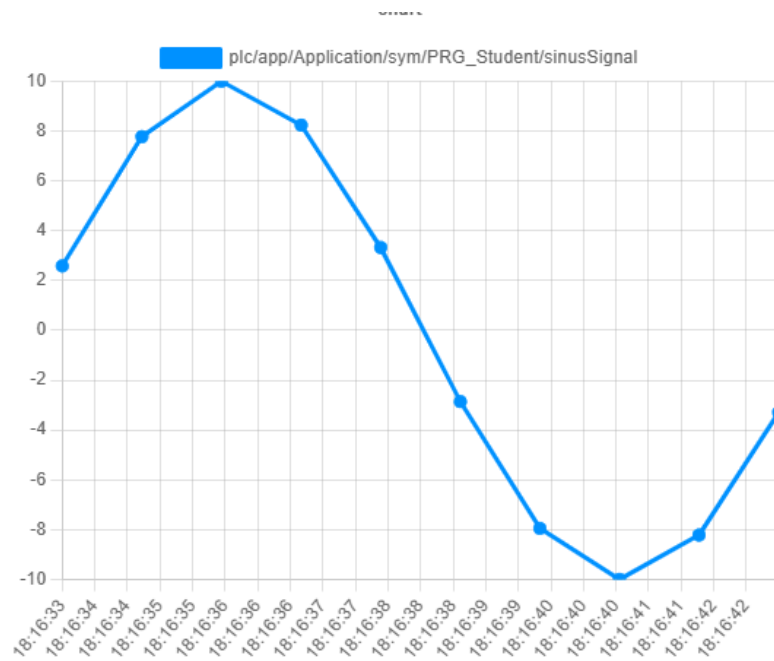
```
END_VAR
```

Compléter le programme Node-RED avec le code ci-dessous



Graphique dans Node-RED

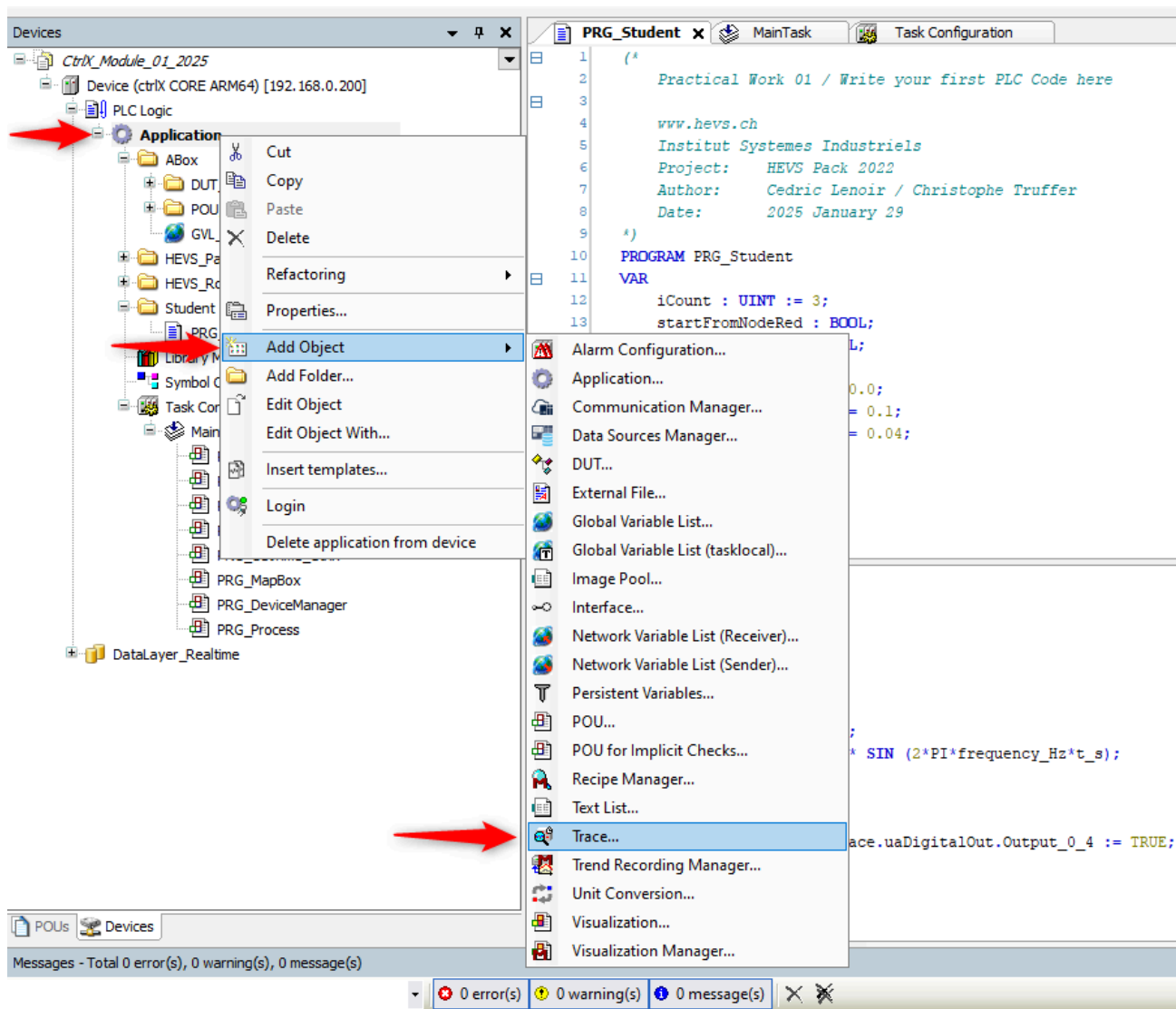
Résultat :



Signal sinusoidal sur Node-RED

Ajouter l'outil "Trace" dans CtrlX PLC

Ajouter l'outil "Trace" dans CtrlX PLC et visualiser le signal sinusoidal.



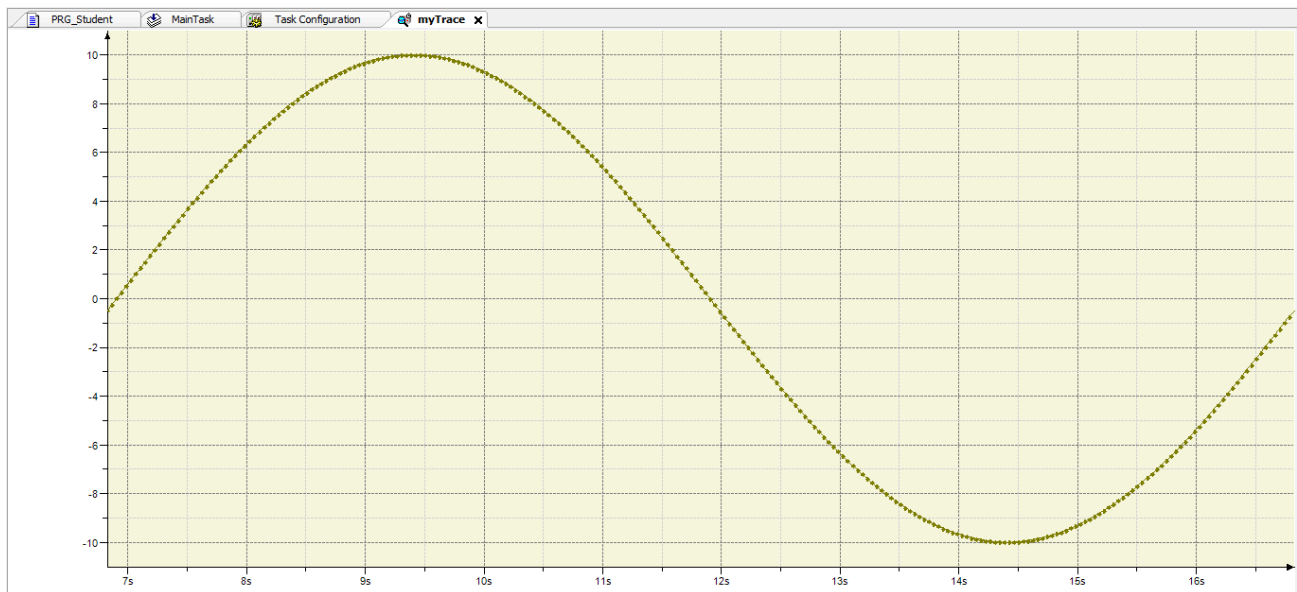
Description

Outil "Trace" dans CtrlX PLC

Pour visualiser le signal :

- Sélectionner la fenêtre "Trace"
- Bouton de droite de la souris
- Sélectionner "Update Trace"

Résultat :



Signal sinusoidal avec l'outil "Trace"

Que constatez-vous si vous comparez le signal sinusoidal affich   sur "Node-RED" avec celui affich   avec "Trace" ?